UC Davis Math Department Course Scheduling Optimization

1. Problem description and dataset source

This project addresses university course timetabling by assigning undergraduate upper-division Mathematics courses (MAT 100 and above) at UC Davis in Fall 2025 to rooms and time slots while satisfying scheduling and capacity constraints.

Our model uses publicly available UC Davis data:

1. **Math Department Fall 2025 Course Listings**
   Upper-division course offering, enrollment sizes, and course times (task 1) was gathered from the Math Department website (https://www.math.ucdavis.edu/courses).
2. **UC Davis Registrar Room Capacity Database**
   Wellman Hall room capacities were obtained from the Registrar's database (https://registrar.ucdavis.edu/faculty-staff/classroom/ga-cap-sorted), ranging from small 25–35 seat rooms to larger 80–100+ seat lecture halls.

Using these datasets, we performed two tasks:

- **Task 1:** Created a feasible timetable where each course is assigned to exactly one room and one time slot, avoiding double-bookings and respecting room capacities
- **Task 2:** Created an optimized timetable that minimizes scheduling courses at inconvenient times, reducing the number of early morning and late afternoon/evening sessions, improving the schedule's practical quality while maintaining all constraints from Task 1

Together, these sources allow us to build a realistic scheduling model that reflects the actual constraints present in UC Davis course planning.

2. Mathematical formulation

We modeled the timetabling problem as a binary integer linear program (ILP). The model assigns each course to exactly one feasible room–time combination while avoiding conflicts and penalizing undesirable scheduling patterns

We use the following sets:

- C: set of upper-division mathematics courses
- R: set of available classrooms in Wellman Hall
- T: set of possible MWF time slots

We define the following parameters:

- enrollment_c: number of students enrolled in course $c \in C$
- capacity_r : seating capacity of room $r \in R$
- timePenalty_t: penalty weight for time slot $t \in T$, reflecting how undesirable that slot is
  - Early and late slots (e.g., 9:00 AM, 3:10 PM, and later) receive higher penalties
  - Midday slots (e.g., 10:00 AM–2:10 PM) receive low or zero penalty
- roomWeight: penalty coefficient for using an additional room

- slotWeight: penalty coefficient for using an additional time slot

To enforce capacity constraints efficiently, we only allow assignments *(c,r,t)* where $\text{enrollment}_c \leq \text{capacity}_r$

We introduce three types of binary decision variables:

- **Course–room–time assignment:**

$$x_{c,r,t} = \begin{cases} 1 & \text{if course } c \text{ is assigned to room } r \text{ at time slot } t, \\ 0 & \text{otherwise,} \end{cases} \qquad \forall c \in C,\ r \in R,\ t \in T$$

  defined only when $\text{enrollment}_c \leq \text{capacity}_r$
- **Room usage:**

$$\text{roomUsed}_r = \begin{cases} 1 & \text{if room } r \text{ is used by at least one course,} \\ 0 & \text{otherwise,} \end{cases} \qquad \forall r \in R$$

- **Time-slot usage:**

$$\text{slotUsed}_t = \begin{cases} 1 & \text{if time slot } t \text{ is used by at least one course,} \\ 0 & \text{otherwise,} \end{cases} \qquad \forall t \in T$$

We included the following constraints:

- **Each course is assigned exactly once:**

$$\sum_{r,t} x_{c,r,t} = 1 \quad \forall c$$

- **No double-booking of rooms:**

$$\sum_{c} x_{c,r,t} \leq 1 \quad \forall r, t$$

- **Link assignment to room/time usage:**

$$x_{c,r,t} \leq \text{roomUsed}_r, \qquad x_{c,r,t} \leq \text{slotUsed}_t \quad \forall c, r, t$$

These constraints ensure we correctly count which rooms and time slots are actually used.

Our objective combines penalties for (1) placing courses in undesirable early/late slots and (2) using many rooms/time slots:

$$\min \sum_{c,r,t} \text{timePenalty}_t \, x_{c,r,t} \ + \ \text{roomWeight} \sum_{r} \text{roomUsed}_r \ + \ \text{slotWeight} \sum_{t} \text{slotUsed}_t$$

The weight values control whether we prioritize minimizing early/late classes or minimizing the number of rooms used.

3. Implementation

For the first task, we constructed the initial conflict-free timetable manually using Google Sheets. This approach allowed us to systematically assign each course to a room, following the time slot of the course, while checking the constraints as we built the schedule.

In the second task, we implemented the timetabling model as a binary integer linear program (ILP) and solved it using the SCIP optimizer. The model was implemented in a standard algebraic modeling framework and solved to prove optimality. The core decision variable assigns each course to a specific room–time pair. Because all variables are binary, the solver must choose a feasible combination that simultaneously satisfies room capacities, avoids conflicts, and minimizes penalty costs. The final model contains several thousand binary variables after preprocessing, reflecting all feasible course–room–time combinations.

We also solved the linear relaxation of the ILP to obtain a lower bound on the optimal objective value. Comparing the relaxation bound to the integer solution provides insight into solution quality and the integrality gap of the formulation.

Preprocessing:

To reduce model size and improve computational efficiency, we applied the following preprocessing steps:

- **Capacity filtering:**
  Assignment variables xc,r,tx_{c,r,t}xc,r,t were created *only* if the enrollment of course ccc does not exceed the capacity of room rrr. This eliminated many infeasible room assignments a priori.
- **Fixed meeting pattern:**
  All courses were assumed to follow a MWF schedule, so only MWF time slots were included in the time set TTT.
- **Restricted building:**
  All classes were restricted to Wellman Hall, significantly tightening the room availability constraints and making the problem more realistic and challenging.
- **Penalty structure:**
  Time slots were assigned penalty weights to discourage early (9:00 AM) and late (3:10 PM and later) classes, while midday times were preferred. Additional penalty terms were included for the total number of distinct rooms used and unfavorable time slots.

Several simplifying assumptions were made to keep the model tractable:

- Courses are independent; student-level conflicts are not modeled
- All sections of a course are treated as independent scheduling units
- Instructor availability and preferences are not included
- All rooms are assumed equally suitable, aside from capacity
- Travel time between rooms is ignored since all rooms are within the same building

Despite these assumptions, the model captures the key structural constraints faced by real academic scheduling problems.

4. Results

Room/ Time	9:00	10:00	11:00	12:10	1:10	2:10	3:10	4:10	5:10
Wellman 1	MAT 135A B01	MAT127C A01				MAT 108 A01	MAT 115A B01		
Wellman 3		MAT 145 A01				MAT 108 A02	MAT 115A B02		
Wellman 5	MAT 135A B02	MAT 150A B01				MAT 127B A01		MAT 150A A01	
Wellman 7		MAT 145 A02			MAT 108 B02				
Wellman 109		MAT127C A02				MAT 127B A02			
Wellman 115		MAT 128C		MAT 129	MAT 147				
Wellman 201			MAT 108 D01						
Wellman 205		MAT 150A B02			MAT 108 B01			MAT 150A A02	
Wellman 207			MAT 108 D02						
Wellman 212					MAT 128A	MAT 118A	MAT 167		
Wellman 229	MAT127A A02			MAT 115A A01					MAT 127A B02
Wellman 230	MAT119A		MAT 185A	MAT 168					
Wellman 233	MAT 127A A01			MAT 115A A02					MAT 127A B01

Table 1: Avoids double-booking and respects room capacities (Task 1)

Room/Time	10:00	11:00	12:10	1:10	2:10
Wellman 109	MAT108 B02	MAT150A B01	MAT127A A02	MAT127C A01	MAT145 A01
Wellman 119	MAT115A A01	MAT115A B01	MAT147	MAT108 A01	MAT185A
Wellman 201	MAT127A A01	MAT108 B01	MAT150A A02	MAT127A B01	MAT127A
Wellman 207	MAT108 D02	MAT150A B02	MAT108 D01	MAT108 A02	MAT115A B02
Wellman 212	MAT129	MAT115A A02	MAT118A	MAT 127C A02	MAT 168
Wellman 230	MAT119A	MAT145 A02	MAT135 B01	MAT127B A01	MAT128C
Wellman 234	MAT128A	MAT167	MAT150A A01	MAT135A B02	MAT127B A02

Table 2: Efficient, minimizes scheduling too many early or late sessions (Task 2)

The final objective value obtained by the SCIP solver is:

Objective Value=152

This value represents the total penalty cost, composed of:

- Room usage penalties (7 rooms × 15)
- Time slot usage penalties (5 slots × 8)
- Early/late scheduling penalties for unavoidable assignments

5. Reflections

Because enrollment sizes force the use of multiple rooms and time slots, some penalty costs are unavoidable. The solver's result reflects the best possible tradeoff between minimizing undesirable class times and maintaining feasibility under tight space constraints.

Overall, the optimized timetable is substantially more student-friendly and resource-efficient than the baseline feasible schedule, demonstrating the value of integer programming for academic scheduling problems.

6. Solver Performance Comparison

The LP Relaxation has a running time of 2 seconds, whereas the ILP has a running time of 1106 seconds. The LP Relaxation allows variables to take on fractional values, making the optimization landscape continuous and much easier to solve. Standard LP solvers can efficiently navigate this smooth objective function without the combinatorial complexity of binary decisions.

The ILP model had a greater running time because it restricts all variables to binary values, creating a combinatorial search problem. The solver must explore different combinations of course assignments while minimizing the objective that penalizes undesirable early/late slots and the number of distinct rooms used. This discrete optimization requires examining many potential solutions (search nodes) to prove optimality, significantly increasing computational effort.

7. ILP vs. LP relaxations Comparison

The two solvers produced significantly different results. The LP relaxation had more flexibility because assignment variables could take on fractional values between 0 and 1, making it computationally easier to solve. This resulted in a lower objective value of 4.417 with 1 solution found after exploring 1 search node in 2 seconds. The model minimized penalty costs by distributing course assignments fractionally across multiple rooms and time slots. For instance, it assigned 0.021 of a course to one room-time pair and another 0.021 to another. While the objective was lower than the ILP result, it is not practical because fractional values cannot occur in real time and cannot represent an actual timetable.

In contrast, the ILP model required binary decision variables where each course required an assignment to exactly one valid room and time slot. This combinatorial constraint dramatically increased the problem difficulty, leading to a greater running time and the exploration of 703 search nodes in order to reach an optimal solution. The resulting objective value was 152 with 18 solutions found in 1106 seconds, representing a realistic schedule. The gap between the LP optimal value (4.417) and the ILP optimal value (152) reveals the integrality gap - the cost of requiring practical, discrete assignments rather than theoretical fractional ones. These results highlight the fundamental tradeoff between computational efficiency and practical implementation in scheduling problems.